PAGE No.
DATE / /

Building LLM is done

Stage 1 : ✓

Now, Stage 2 : Training

~~Pretraining~~ : Basics

i) How to measure, our LLM working bad or good

quantitative intuition of not of knowing that it does not look into a quantitative matric,
& that quantitative matric is generally Loss function

So, Next stage consist

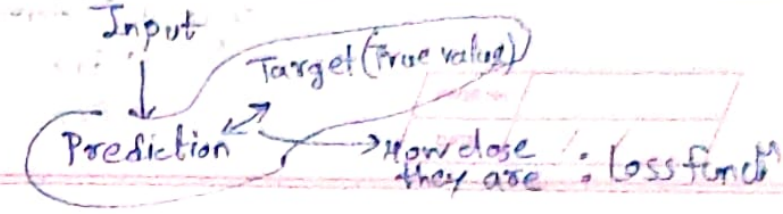
done already

1] Text generatn

2] Text evaluatn

3] Training & Validation losses

LLM training functn → Text generatn Strategies → Weight saving & loading → Pretrained weight from OpenAI



1] Inputs & targets

```
inputs = torch.tensor([[16833, 3626, 6100],   # "every effort moves"
                       [40, 1107, 588]])      # "I really like"
```

True Predict^n

```
target = torch.tensor([[3626, 6100, 345],   # "effort moves you"
                       [107, 588, 11311]])  # "Really like chocolate"
```

here 588 represent 'like'

means, it predict what comes after 'I Really _ _ '

same Foo every number in target tensor

2] Outputs

input → (GPT Model) → output token IDs

Tokenizer → (tokenID → logits) → (logits → tokenID)

Just Revise Last Few Pages

~~logits → token ID~~

input → logits ([[0.1 0.2 0.1 0.3 0.1] # every
[...] # efforts
])

#Position 2 — Ind. posi^n with highest probability

→ Index of highest Probab → token id

b) Gives ~~target~~ index of target token from the given tensor / array

So, if

If 'every' is input, the token of token id 16657 will be output.

Token ID
tensor([[[16657],
[339],
[42826]], } Batch 1

[[49906],
[29669],
[41751]]], } Batch two

These are Output Token IDs

&

We want, it

As much as closer to target (true) Matrix.

3) Finding loss between target & output

See,

Inputs → Tokens → Probability Matrix → Target Matrix

every → 2 → [... 0.14, 0.13, 0.12, 0.15, 0.18 ...] (index=4, efforts) → [0.13],

effort → 3 → [... 0.12, 0.14, 0.13, 0.15] (index=112, moves) → [0.13],

moves → 4 → [... 0.11, 0.2, 0.21, 0.11, 0.11 ...] (index=341, You) → [0.2],

TargetMatrix: [4, 112, 341] ← Position of prob

Target Matrix: [4, 112, 541] ——— (1)

↳ Probability at that index Not highest yet because, Not trained/optimised yet

So
So lets just take probabilities at that indices

↳ [0.13, 0.13, 0.2]

↳ [i_{11}, i_{12}, i_{13}]

So if there are two batches, ↳ Every effort moves "I am a"
i.e. two logits
i.e. two Target Matrix → [4, 112, 541]
→ [10, 12, 71]

~~Sou~~Suppose probabilities at given indexes are

Batch 1 : [i_{11}, i_{12}, i_{13}]
Batch 2 : [i_{21}, i_{22} i_{23}]

& say
probalities, are like $i_{11} \longrightarrow P_{11}$ so,

0.13 0.13 0.2

Batch 1 : [P_{11}, P_{12}, P_{13}]
Batch 2 : [P_{21}, P_{22}, P_{23}]

↓

Merge them

[P_{11}, P_{12}, P_{13}, P_{21}, P_{22}, P_{23}

P_{23} ↳ probability of word 'a' coming after 'I, am'

Goal of training is to get these value close to 1

Cause they are target value
↳ So the Probability is highest

Steps to make

1] Logits

↓ (softmax)

2] Probabilities

↓

3] Target probabili

i_1, i_2, i_3, i

↓

4] Log probabili

↓

5] Average log Probability

↓

6] Negative ~~average~~ log probabi

↓

Cro

it closer to 1

[[0.1113, -0.1051, 0.3666, ...]]

[[1.8849e-05, 1.5117e-05, 1.6876e-05]]

ies [7.4541e-05, 3.1061e-05, 1.5632e-05] → Merged target Probability

→ log of

ies [-9.5042, -10.3796, -11.3677, ...]

= -10.7722

The negative avg log probability is the so-called loss we want to compute.

by = 10.77722

ss Entropy Loss'

This Measures difference between 2 probability Distribution

So, have, $[P_{11}, P_{12}, P_{13}, P_{21}, P_{22}, P_{23} \ldots]$

↓ taking log

$[\log(P_{11}), \log(P_{12}), \log(P_{13}), \log(P_{21}) \ldots]$ 1 x n

↓ Mean of this

↓

$$\text{Mean} = \frac{\log P_{11} + \log P_{12} \; - - - \log P_{23} \ldots}{n}$$

↓ Negative of mean

↓ Negative log likelyhood

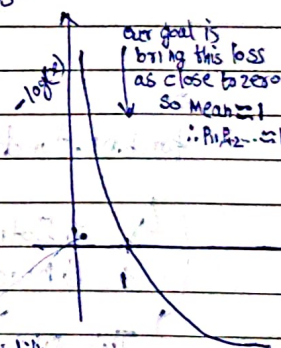
$$= \frac{-\Sigma \log(P_{11}) + - - - \log(P_{23}) \ldots}{n}$$

We want this Negative log likelyhood as less as possible

∴ cross entropy loss as low as possible as close to zero

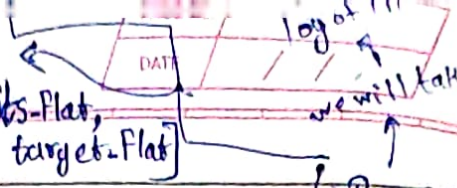
↳ It ensures that the indices which we choosed from target matrix have highest prob chances of selection

Loss =



Initially we are taking $(\log p_i)$ log of P_i so it will somewhere at point ' ' our goal is to bring it near to 1 ∴ pushing Right side

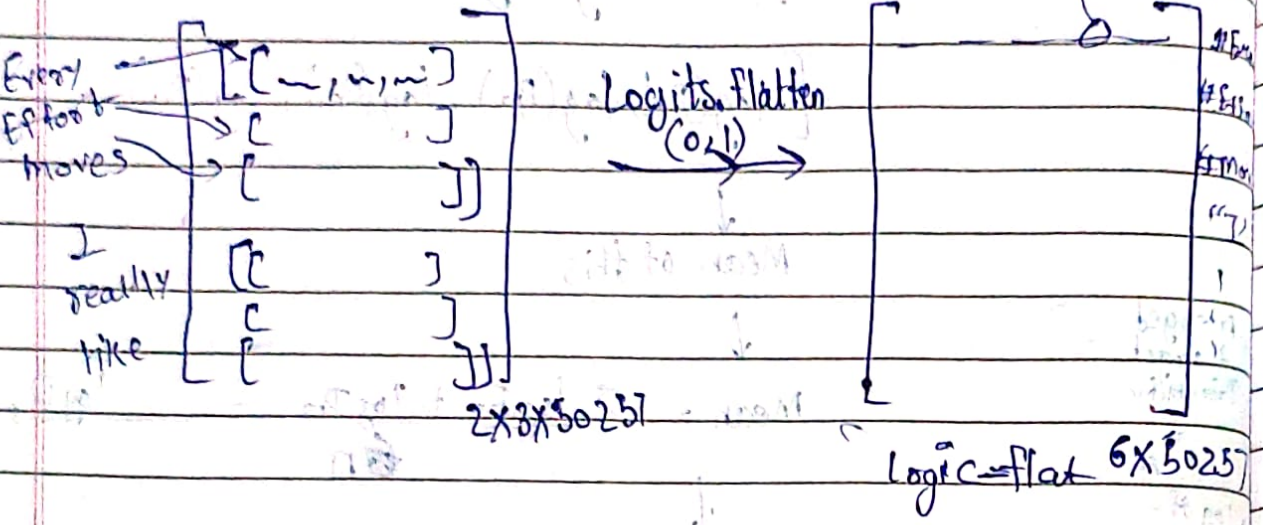
↳ ~~torch.nn~~ torch.nn.functional.cross_entropy[logits_flat, targets_flat]



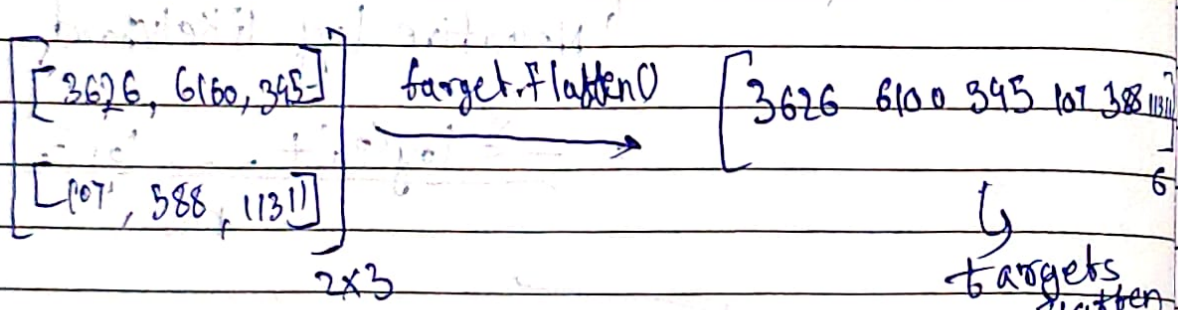
So, see,

$P_{11} = 3626$

★ Logits



★ Targets



So,

torch.nn.functional.cross_entropy[logits_flat, targets_flat]

↳ ① Softmax of logits
↳ ② Negative log likelyhood

This one line, will give cross entropy loss between output & target

The goal is to make this average log probability as large as possible by optimizing the model weights.
Due to this log, largest possible value is 0, & we are currently far away from 0.

In Deep learning, instead of maximizing the average log-probability, it is a standard convention to minimise the negative average log probability instead of maximizing -10.7722 so that it approaches 0; in deep learning, we would minimise 10.7722 so that it approaches 0,
The value negative of -10.7722, i.e, 10.7722, is also called cross-entropy loss in deep learning.

Perplexity : Another loss measure like cross Entropy.

- Measure how well the probability distribut" predicted by the model matches the actual distribution of words in the dataset
- More interpretable way of understanding model uncertainty in predicting next token
- Lower perplexity score = Better prediction

Perplexity = e^{loss} (exponent of loss)
= torch.exp(loss) ≃ 48725

PAGE No.
DATE / /

Suppose
torch.exp(loss) = 48725

Model is roughly as uncertain as if it had to choose the next token randomly from about 48725 tokens in the vocabulary

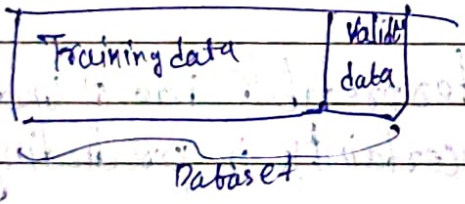
Much more Interpretable

3] Training & Validation losses

1) ~~to~~ tokenizath

2) Divide the dataset into training & Validath (Testing purp

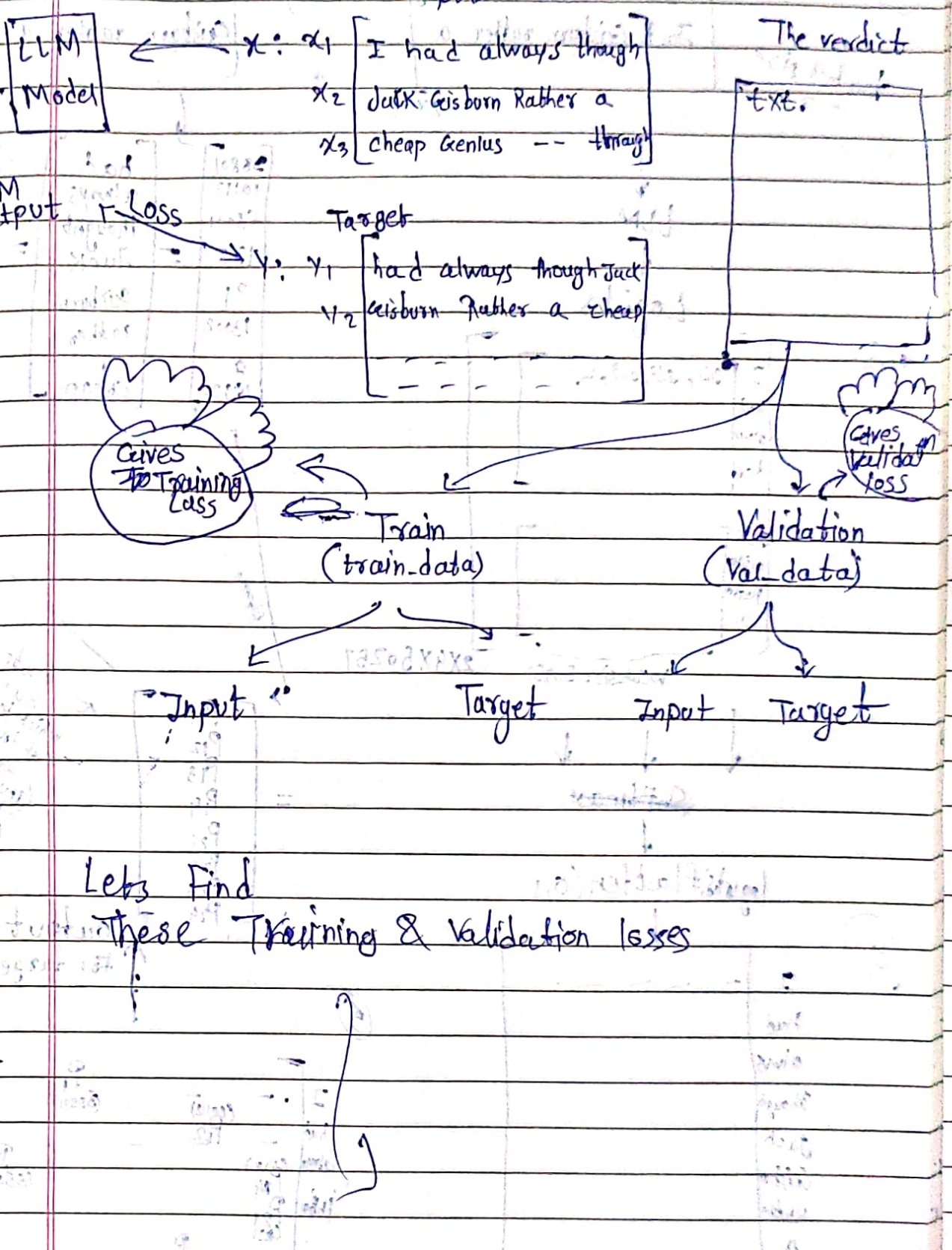
```
train_ratio = 0.90
split_index = int (train_ratio * len(text_data))
train_data = text_data [:split_index]
Val_data = text_data [split_index:]
```



3) Use dataloader to chunk training & validation data into input & output datasets

Context_len=4=stride

So we have



Lets Find
These Training & Validation losses

PAGE No.
DATE / /

Input

(1) $x =$ [I had always thought / Jack Gisburn rather a] — 2 × 4

batch size = 2

(2) Target

$y =$ [had always thought / Gisburn rather a cheap]

LLM

Logits

(3)

I	o_{11},	o_{12},	o_{13},	...
had				
always				
thought				
Jack				
Gisburn				
Rather				
a				

Vocab size = 50257

2 × 4 × 50257

Softmax

logits.flatten(0,1)

(4)

I	- - - - - -
had	
always	
thought	
Jack	
Gisburn	
Rather	
a	

8 × 50257

Softmax

38801	had
10112	always
55001	thought
1338	= Jack
21	Gisburn
13002	rather
8	a
18920	cheap

Probs

P_{11}	
P_{12}	
P_{13}	
P_{14}	=
P_{21}	
P_{22}	
P_{23}	
P_{24}	

output of our model which tell probability of true/target token (This is not reproduce highest probability because model is not trained yet)

output probability for target token ID

(5)

I	...	P_1 (38801)	...
had	(P_{12} (10112))		
always	(P_{13} (55001))		
thought	P_{14}		
Jack	P_{21} (1338)		
Gisburn		P_{22} (13002)	
Rather			
a	P_{23} (8)		P_{24} (18920)

Probability tensor

$$\begin{bmatrix} P_{11} \\ P_{12} \\ P_{13} \\ P_{14} \\ P_{21} \\ P_{22} \\ P_{23} \\ P_{24} \end{bmatrix} \xrightarrow{\text{Cross entropy loss}}$$

$$\text{total\_loss} = -\frac{1}{4}\left[\log P_{11} + \log P_{12} + \log P_{13} + \log P_{14}\right] + -\frac{1}{4}\left[\log P_{21} - - - + \log P_{24}\right]$$

$$\frac{\text{total\_loss}}{\text{No. of batch}} = \text{loss per batch}$$

So we got loss for input, it is same for the Training loss & validation loss method

bot ; (90% of dataset) (10% of dataset)

(overall loss ↑)

(loss per batch = ↑ x) (loss per batch = x)

Code in Vscode

P_i is ~~token~~ Probability of token 38801 which is had 'For. token 'I'

4] LLM Pretraining loop

↳ Objective is, Minimize the loss
i.e. Bring output closer to target

LLM Pre-traing loop schematic

one epoch is one complete pass over a training set

1) For each training epoch

2] For each batch in training set

3] Reset loss gradients from previous epoch

4] Calculate loss on current batch

5] Backward pass to calculate loss gradients

6] Update model weights using losse gradients

7] Print training & Validat" set losses

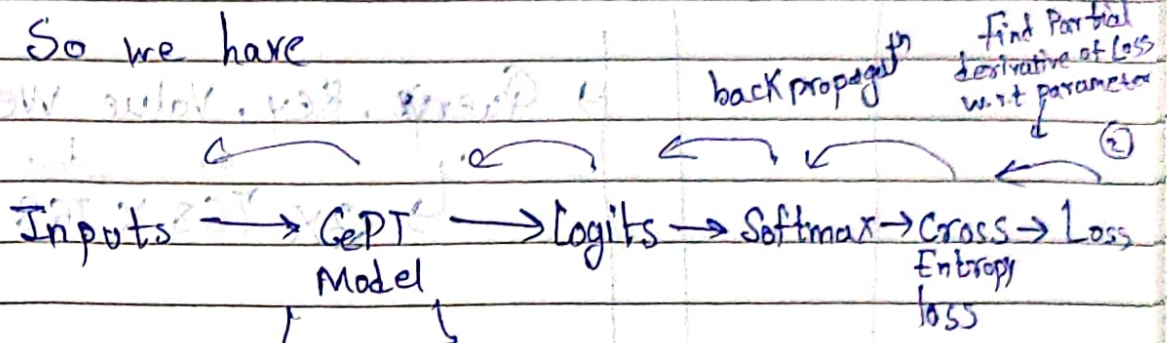
8] Generate sample text for visual inspect"

These are the usual steps used for training deep neural network in Pytorch

optional step for tracking training process

Main step: Find loss gradients using
loss.backward()

So we have



↳ Most parameter comes in Model

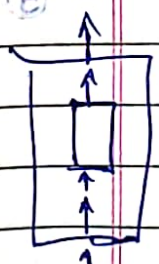
by this oneline

↳ loss.backward()

↳ We are essentially finding the gradient of loss with respect all of the parameters in model

$P_{new} = P_{old} - \alpha \frac{dL}{dP_{old}}$

+ optimizer.step: updating all these values



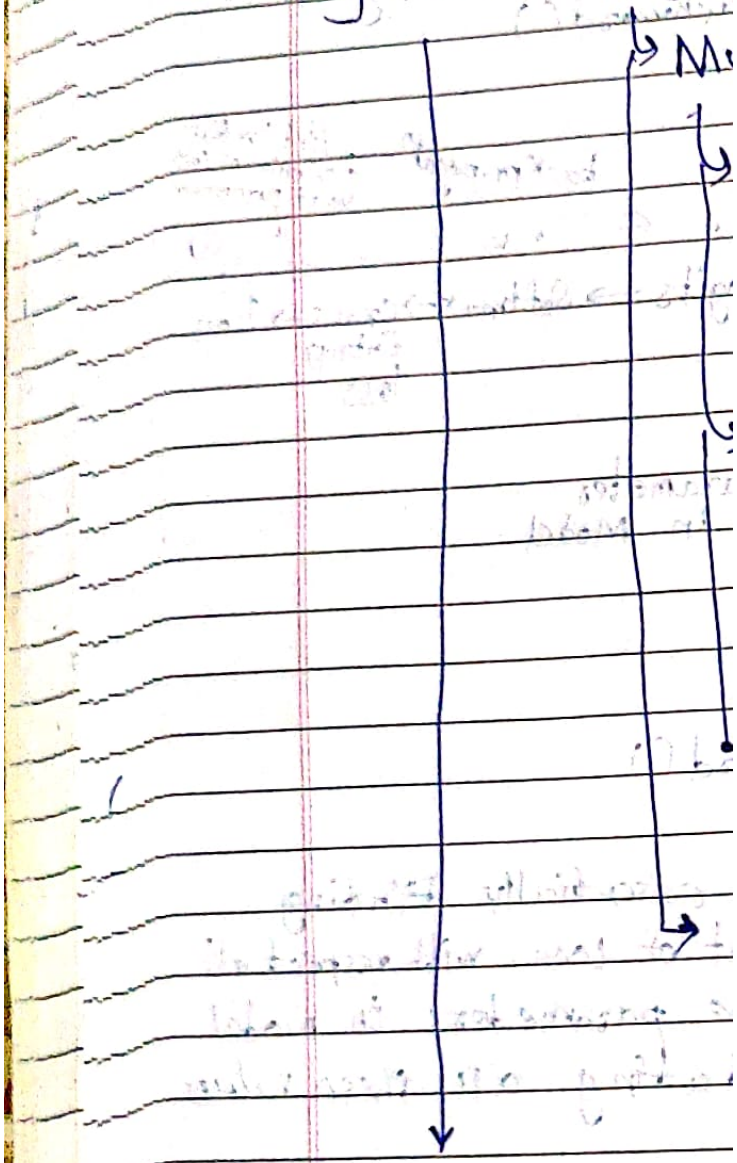
So, lets understand
No. of parameter we using,

1] Embedding Parameter

Token-embed. parameter		Position Embedding Param (context len)
50257 X 768	+	1024 X 768

= 38.4M parameter —— (1)

2) Transformer



lock parameter :-

ltihead Attention

Querie, Key, Value weights

↳ $3 \times \underset{d_{in}}{768} \times \underset{d_{out}}{768} = 1.77M$ — (2)

Output head

↳ $768 \times 768 = 0.59M$ — (3)

→ Total = 2.36 M — (4)

Feed Forward Neural Newor

↳ $\underset{\text{Expansion}}{768 \times (4 \times 768)} + \underset{\text{Contraction}}{768 \times (4 \times 768)}$

= 4.72 M — (5)

(4) & (5))

.36 + 4.72 M)

M — (6)

3] final layer (Softmax output)

↳ $768 * 50257 = 38.4$ M — (7)

∴

Total Number of parameters, to optimized

⇒ (1) + (6) + (7)

= 38.4 + 85.2 + 38.4

≃ 162 M

Bot in GPT2 they used 124M

The reason is weight tying,

↳ In output layer project layer, they recycle the same No. of embedding Parameter in embeding layer

For each of this parameters, we need to perform gradient descent updates.

First we need gradient, which we get in step 5 of pretraing LLM schematic

once we, get that we update it

$$W = W - \alpha \frac{dL}{dw}$$

update model weight in each iterat

we calculate $\frac{dL}{dw}$ in Backward pass

as we do,

α : step size

$$W = W - \alpha \frac{dL}{dt}$$

the, loss function, goes on decreasing, & reach some sort of minima.

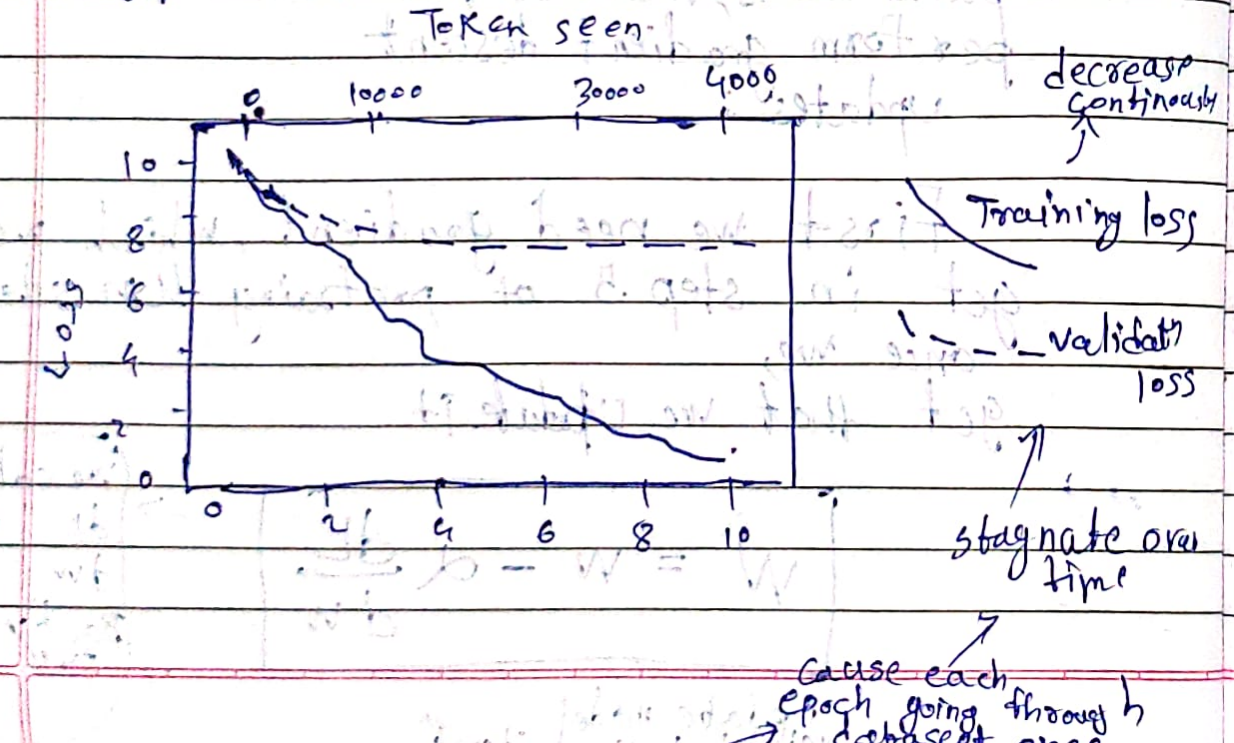
we, use

AdamW to update & optimse parameters

↳ "works very well

code in vs code

but



Both training & validation losses start to improve for the first epoch, However, the losses start to diverge past the second epoch.

↳ Cause the model is Overfitting to training data.

Now, Model is just memorizing the training data so it is just searching & generating text snippet

↳ This ~~is~~ Memorization is expected since we are working with a very, very small training dataset & training the model for multiple epochs.

↳ This Memorisation is sign of overfitting

Usually, it's common to train a model on a much larger dataset for only one epoch

Until now, our model is just memorising & generating

Decoding Techniques to control Randomness

Until Now, the generated token is selected corresponding to the largest probability score among all token in the vocabulary

↳ Leads to a lot of randomness & diversity in generated text.

There, are 2 techinques to control Randomness

- temperature scaling
- top-k Sampling

greedy decoding

Temperature Scaling → it gets finds the index with highest Probab. score.

↳ Replace argmax with probability Distribution

Probability Distribution

↳ Multinomial probability distribution sample next token according to probablity score.

So, basically we sample Multiple possibilities.

∴ We are going to sample next token with Multinomical Distribution

It samples the next token proportional to its probablity scores,

lets say

'Every step moves you'

↓

LLM

↓

Logits

↓

Softmax

↓

Multinomial Dis

↓

'Forward'

It require Multiple running

'forward' is still the most likely token & will be selected by multinomial most of the time but not all time.

So, if we use Multinomial

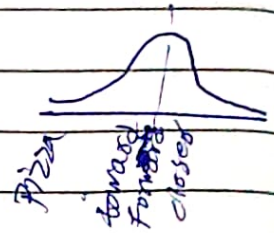
↓

Sometimes it will chose

'closer' 'forward' 'toward' 'inches'

(mostly choose this)

but it also gave chances to other



Temperature: Fancy term for dividing the logits by a number greater than zero

logits

↓

Scaled logits = logits / Temperature

If temp < 1 → Sharper Distribution [0.0, 0.1, 0.0, 0.991, 0.0]

If temp > 1 → Flattened Distribution (More variety, but more Nonsense) [0.37, 0.421, 0.63, 0.123]

Temperature → Just a number

↓

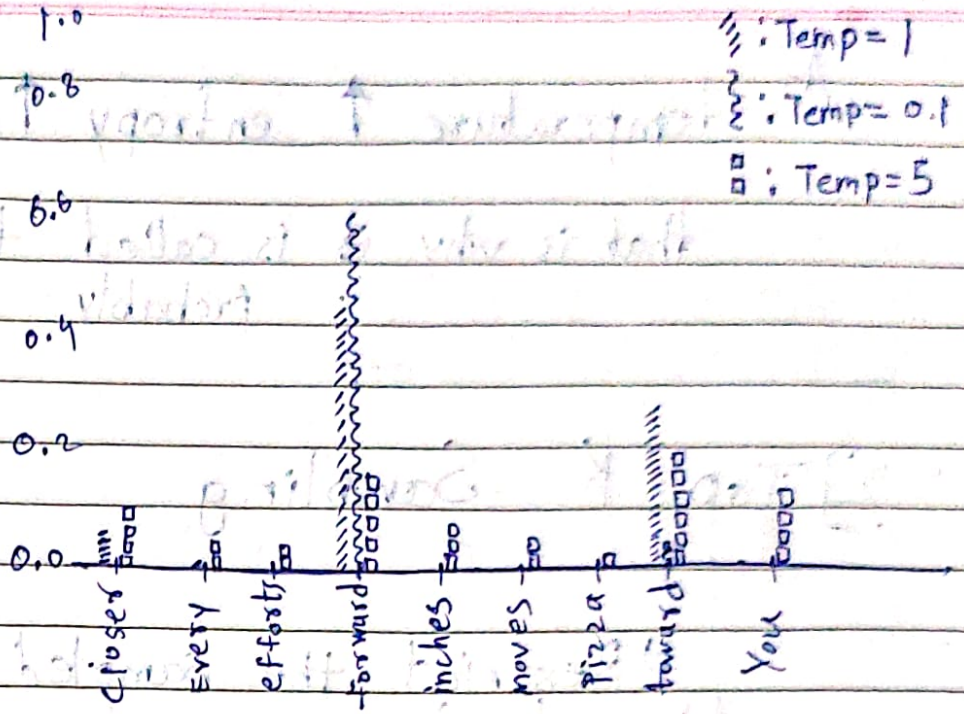
Softmax

↓

Tensor of probability

↓

Multinomial Distribution



Keeping → Temperature : 0.1

↳ It will select 'forward' mostly, approaching behavior of argmax
∴ No chance to other

→ Temperature : 5

↳ result in more uniform distribution where other token select more often But can lead to non-sence selection

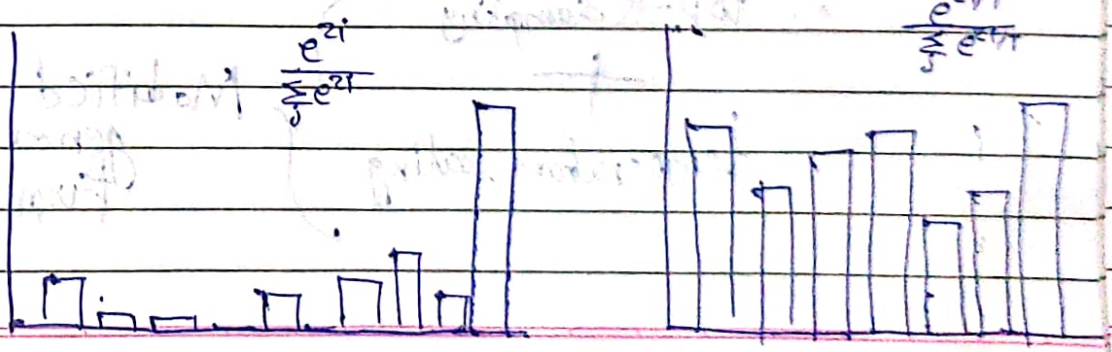
If temp. → ∞ Everyone have probab = 1

Softmax without Temperature (Temperature = 1)

$\frac{e^{z_i}}{\sum_j e^{z_j}}$

Softmax with temperature

$\frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$



less Entropy —Increase in entropy with increase in T→ More Entropy

As temperature ↑ entropy ↑

that is why it is called temperature probably

2] Top-K Sampling

↳ Restrict the sampled token to the ~~topic~~ top K likely token & exclude all other

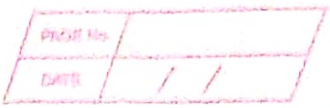
logits = [9.51, 0.89, 0.75, -1.90, 6.75, -1.89, 6.28]

Top-K (K=3) = [9.51, 0.89, 0.75, -1.90, 6.75, -1.89, 6.28]

inf mask = [9.51, -inf, -inf, -inf, 6.75, -inf, 6.28]

Softmax = [0.06, 0, 0, 0, 0.57, 0, 0.36,]

∴ top-K sampling + temperature scaling } Modified text generation Function.



Logits

↳ Top-K

↳ Logits / Temp

↳ Softmax

↳ Sample From Multinomial

∴ This ~~ensure~~ reduces Overfitting.

& Now, our model Not just Memorize the ~~text~~ From .txt, but also change those words.

6) Weight Saving & loading

Let us learn to save & load a pretrained model

1) torch.save(model.state_dict(), "model.pt")

dictionary mapping each layer to its parameter

filename in which we are going to store

instead of running New instance of our model, everytime, we can save them in file

in Pytorch, the learnable parameters (weights & biases) of a torch.nn.Module model are contained in the model's parameters (accessed with model.parameters()). A state_dict is simply a python dictionary object that maps each layer to its parameter tensor.

we can use any, for storing

2] model.load_state_dict(torch.load("model.pth"))

↳ Someone who is working on next day or next pc, they can load the pretrained model, using above command

∴ their model will be loaded with the parameters in original model stored in file!

∴ Loads a model's parameter dictionary using a deserialized state_dict.

So, just by creating instance,

ex:
```
model = GPTModel(GPT_CONFIG_124M)
model.load_state_dict(torch.load("model.pth"))

model.eval()   # This is used to see parameter
               # Nothing to do with loading or saving
```

This will save parmeters,

3] Saving optimizer state is also recommended

↳ optimizer.state_dict()

It contains: "hyperparameters → historical data, such as past gradients

Adaptive optimizers such as AdamW stores additional parameters for each model weight. AdamW uses historical data to adjust learning rates for each model parameter dynamically.
Without it, the optimizer resets, and the model may learn sub-suboptimally or even fail to converge properly, which means that it will lose the ability to generate coherent text.
Using torch.save, we can ~~have~~ save both the model and optimizer state_dict contents as follows:

```
torch.save({"model_state_dict": model.state_dict(),
            "optimizer_state_dict": optimizer.state_dict()
           },
           "model_and_optimizer.pth"
)
```

&

we can load this by

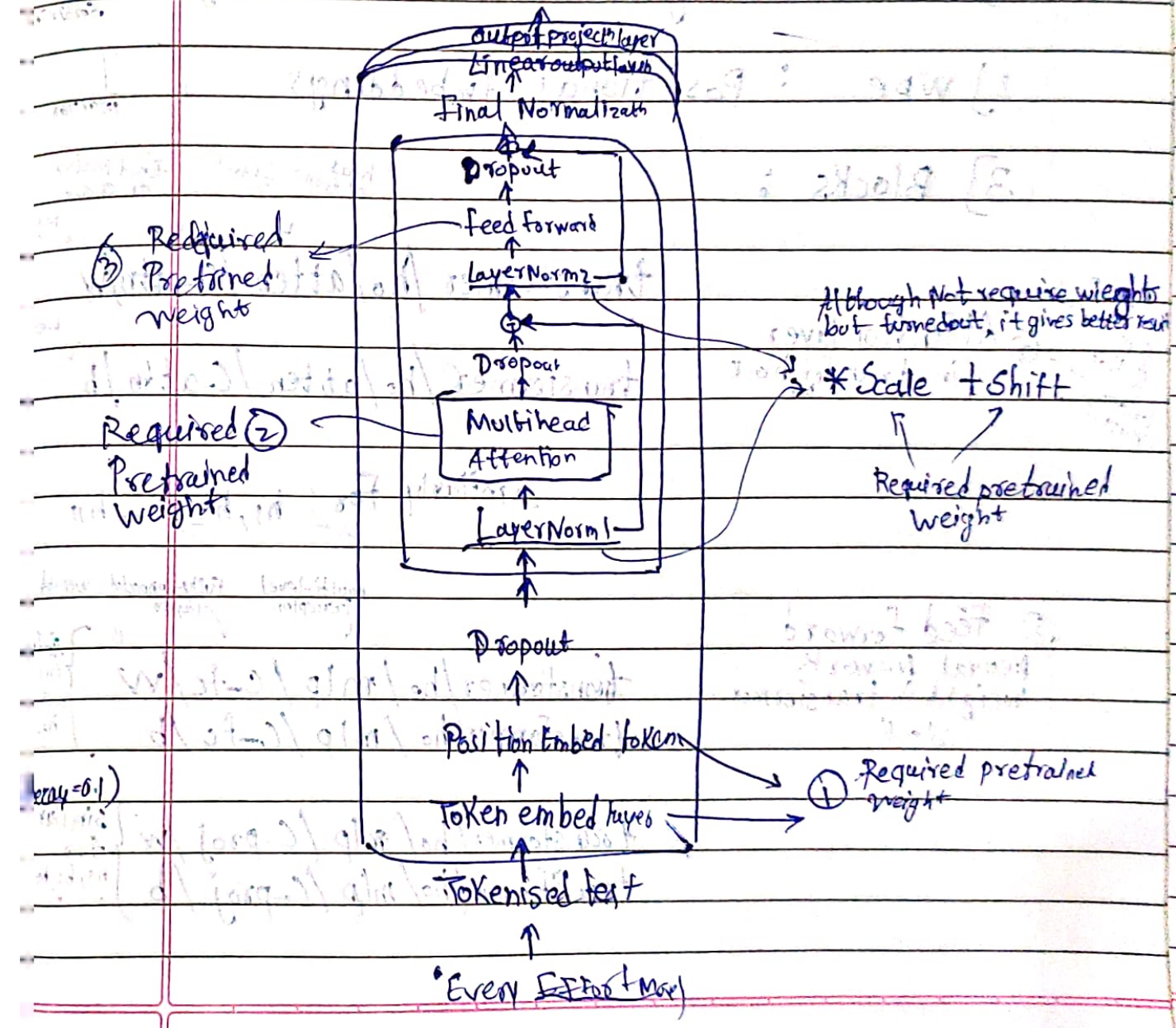
```
checkpoint = torch.load("model_and_optimizer.pth")
model = GPTModel(GPT_CONFIG_124M)
model.load_state_dict(checkpoint["model_state_dict"])
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-4, weight_d
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
model.train()
```

7] Pretrained Weights from OpenAI

① OpenAI originally saved the GPT-2 weights via Tensorflow, which we have to install

Moreover,

One more library called tqdm, which will show download progress.



So, when we download the pretrained weights of gpt-2 from Kaggle we get some files & we load the data from them and their is dictionary obtained called,

Parameter Dictionary.

Parameter Dictionary Keys:

1] wte : Weights for Token embeding [ ] 50257x768

2] wpe : Positional Embeddings [ ] 1024x768

3] Blocks :

① Attention layer in transformer block

in gpt2 they fused Q,K,V under one matrix

head of transf | Attention | Fused Matrix of Q,K,V

transformer/h0/atten/c_attn/w

bias

transformer/h0/atten/c_attn/b

Similarly for h1, h2, ... h11.

② Feed forward Neural Network weights in transformer block

multi-level Perceptron | Fully-connected layer | weights

transformer/h0/mlp/c_fc/w
transformer/h0/mlp/c_fc/b

similar for h1, h11, h2..

transformer/h0/mlp/c_proj/w
transformer/h0/mlp/c_proj/b

similar for h1, h2, b2 ...

Output projecth layer — transformer/ho/attn/cproj/w

similar for h1, h2, h3

4] Layer Normalizath

Layer Norm1:
- transformer/ho/ln_1/g → Layer Norm Scale
- transformer/ho/ln_1/b → Layer Norm Shift

Layer Norm2:
- transformer/ho/ln_2/g
- transformer/ho/ln_2/b

similar For h1, h2 .. hn

∴ With the 'Blocks' Key, we are getting
i) Attenth layer weights
ii) Feed forward Weights
iii) Output projecth weights
iv) Layer Norm weights

4] g : weights of Final Normalizath Scale

5] b : ~~weig~~ bias weights of Final Normalizath Shift.

PAGE No.
DATE / /

In your code,

'params' will be this parameter Dictionary
'setting' will hold the CONFIGURATION

Stage1 Stage2

Done